

# iQOO Wellness AI Engine

## OEM-Ready Camera Intelligence Platform — Product Requirements Document

**Version:** 2.0   **Target:** iQOO Hackathon 2026   **Platform:** Android / iQOO / OriginOS-compatible architecture

### 1. MASTER OBJECTIVE

Build a lightweight, on-device Wellness AI Engine designed to eventually sit inside the native iQOO Camera / OriginOS ecosystem. Exactly three capabilities: Food Nutrition Intelligence, Walking & Activity Intelligence, and Real-Time Exercise Posture Correction.

### 2. CRITICAL ARCHITECTURAL TRUTH

A normal third-party APK cannot simply inject into or replace proprietary iQOO Camera internals. The hackathon therefore has two layers: (A) a working CameraX/Camera2 prototype on the iQOO phone and (B) an OEM production architecture in which the native Camera calls a WellnessManager/System Service. Never claim proprietary OriginOS modification without OEM access.

### 3. CORE DESIGN PRINCIPLE

Treat the Camera as an input source. The Wellness Engine owns AI inference, food recognition, personalization, pose analysis, exercise rules, activity calculations, local persistence and privacy. Camera integration owns camera lifecycle, preview, frame delivery and UI overlays.

### 4. REQUIRED HIGH-LEVEL ARCHITECTURE

Camera Integration → WellnessManager → WellnessEngine →

Scene/Food/Posture/Activity/Personalization/Inference/Storage. The engine must remain independent of UI and camera implementation.

### 5. CASCADED AI INFERENCE

Do not continuously run heavy food and pose models whenever the Camera is open. Use a lightweight scene/intent classifier: Food → Food pipeline; Person/exercise → Pose pipeline; Normal scene → no heavy inference. This reduces battery, thermal load and latency.

### 6. FOOD NUTRITION INTELLIGENCE

Identify food from the camera, estimate/accept quantity, retrieve nutrition data locally, and personalize future estimates from local history. Repeated dishes should reuse learned typical quantities.

### 7. FOOD PIPELINE

Camera frame → food/scene detection → food recognition → food ID → portion/quantity → nutrition database → local user retrieval → personalized nutrition → camera UI.

### 8. PERSONALIZATION / RAG RULE

RAG does not retrain the vision model. Keep the model fixed and retrieve user-specific history such as previous food, quantity, preferences and nutrition. Room/SQLite structured retrieval is sufficient for the MVP; embeddings are optional.

### 9. FOOD DATABASE

Local offline data must contain at minimum: food\_id, name, category, serving\_unit, default\_grams, calories, protein, carbohydrates, fat and fiber.

## 10. WALKING / ACTIVITY INTELLIGENCE

Provide steps, estimated calories burned, distance, average speed and activity/walking duration. Persist only steps and calories for 21 days; temporary distance/speed/duration may be computed and discarded.

## 11. ACTIVITY ARCHITECTURE

Prefer low-power Android step sensors / Health Connect. Do not continuously use GPS. Use reliable source data when available and label estimates as estimates.

## 12. REAL-TIME POSTURE CORRECTION

Support a maximum of 10 exercises. Use one lightweight pose model, with exercise-specific rules. Prioritize reliable exercises such as squat, push-up, bicep curl, lunge and shoulder press before expanding.

## 13. POSTURE PIPELINE

Camera → low-resolution frames → pose model → body landmarks → joint angles → exercise state machine → exercise-specific rules → form evaluation → real-time feedback. Feedback must be explainable from measurable landmarks/angles.

## 14. AI MODEL REQUIREMENTS

Use mobile-compatible lightweight models, quantization where appropriate, low-resolution inference, model reuse and hardware acceleration when available. Candidate runtimes include ONNX Runtime, LiteRT/TensorFlow Lite, MediaPipe and Qualcomm/on-device runtimes where confirmed.

## 15. CAMERA INTEGRATION

Prototype with CameraX unless Camera2 is specifically required. Understand Preview, ImageAnalysis, ImageCapture, lifecycle, ImageProxy and YUV frames. Architecture must allow an OEM Camera to provide an equivalent analysis stream.

## 16. WELLNESS ENGINE API

Maintain a clean API boundary such as WellnessEngine.analyzeScene(), analyzeFood(), analyzePose() and getActivitySummary(). The Camera/UI must not depend on internal ML implementation.

## 17. OEM-READY SYSTEM SERVICE CONCEPT

Future production architecture: System Server → WellnessManagerService → WellnessEngine. Binder/AIDL should define the inter-process boundary. The hackathon prototype need not have privileged OriginOS access.

## 18. ANDROID OS CONCEPTS — LAYERS

Understand Application → Android Framework → System Services → Native/HAL → Linux Kernel → Hardware. This project primarily operates around the Application, Framework, System Service, Camera Framework and AI Runtime layers.

## 19. ANDROID OS CONCEPTS — SYSTEM SERVICE

UnderstandSystemService as an OS-level service managed by Android system\_server. WellnessManagerService is the conceptual future gateway to the engine.

## 20. ANDROID OS CONCEPTS — BINDER/AIDL

Binder is Android's IPC mechanism; AIDL defines Binder interfaces. Conceptually Camera/System App → Binder → WellnessManagerService → WellnessEngine.

## 21. ANDROID OS CONCEPTS — PRIVILEGED APPS

Normal APKs and system/privileged components have different security capabilities. Production integration requires appropriate OEM signing, permissions and system-image integration; never bypass security.

## 22. ANDROID OS CONCEPTS — SELINUX

Android security includes SELinux policy. Manifest permissions alone do not grant arbitrary system/hardware access. Do not disable SELinux as a shortcut.

## 23. AOSP CONCEPT

AOSP is the open-source Android base that can be modified and rebuilt. Conceptual flow: AOSP source → framework/system changes → Android build → system image → test. Use AOSP knowledge to design OEM integration; do not claim proprietary OriginOS modification.

## 24. REQUIRED PROJECT STRUCTURE

Use modular folders: app/, wellness-engine/, models/, data/, future-oem/, docs/. Within wellness-engine: scene/, food/, posture/, activity/, personalization/, inference/, storage/. Keep future OEM integration isolated.

## 25. DATABASE

Use Room/SQLite locally. Core entities: users, preferences, foods, nutrition, food\_history, portion\_history and daily\_activity. Only the latest 21 days of persistent activity records are retained.

## 26. PRIVACY

Camera frames remain local and should be discarded after inference unless explicitly needed for a local feature. No cloud backend is required. Personal food/activity history remains on-device.

## 27. PERFORMANCE / THERMAL STRATEGY

Targets: food inference under ~300 ms where hardware permits and posture around 10–15 AI FPS. These are engineering targets, not claims. Lazy-load heavy models, reuse buffers, control FPS and benchmark on the actual iQOO device.

## 28. DEVELOPMENT & TESTING ORDER

Build foundation → camera pipeline → WellnessEngine API → food → activity → posture → personalization → cascaded inference → OEM abstraction → optimization → testing. After each phase: build, test, fix and verify before continuing.

## 29. DEFINITION OF SUCCESS

A reliable iQOO-device prototype demonstrates Camera → Wellness Engine → Food/Pose/Activity on-device. The codebase also clearly exposes the future OEM path: Native Camera → WellnessManager → WellnessManagerService → WellnessEngine, without falsely claiming proprietary OriginOS access.

# Appendix A — Recommended Project Structure

```
iqoo-wellness/  
  ■■■ README.md  
  ■■■ PRD.md  
  ■■■ docs/  
    ■ ■■■ ARCHITECTURE.md  
    ■ ■■■ OEM_INTEGRATION.md  
    ■ ■■■ ANDROID_OS_CONCEPTS.md  
    ■ ■■■ TEST_PLAN.md  
  ■■■ app/  
  ■■■ wellness-engine/  
    ■ ■■■ scene/  
    ■ ■■■ food/  
    ■ ■■■ posture/  
    ■ ■■■ activity/  
    ■ ■■■ personalization/  
    ■ ■■■ inference/  
    ■ ■■■ storage/  
  ■■■ models/  
    ■ ■■■ food/  
    ■ ■■■ pose/  
    ■ ■■■ scene/  
  ■■■ data/  
    ■ ■■■ nutrition/  
  ■■■ future-oem/  
    ■■■ WellnessManager.aidl  
    ■■■ WellnessManagerService.kt  
    ■■■ OEM_INTEGRATION_NOTES.md
```

# Appendix B — Core Data Flow

